

Contents

guide_Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity **1**

- 0. 这篇论文为什么重要 1
- 1. 当时的技术语境 2
- 2. Switch layer 放在哪里 2
- 3. Router 数学 3
- 4. 为什么 top-1 能更高效 3
- 5. Expert capacity 和 overflow 4
- 6. Load balancing loss 4
- 7. Switch layer 的数据流 5
- 8. 训练稳定技巧 6
- 9. 实验证据链 6
- 10. 为什么参数多但 FLOPs 不同步增加 7
- 11. 本地代码怎么对应论文 7
- 12. 与后来 MoE 的关系 8
- 13. 局限和开放问题 9
- 14. 对今天的意义 9
- 15. 常见误解 9
- 16. 闭卷检查 10
- 17. 用 AI agent 学这篇的正确方式 10

guide_Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity

原论文: Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity
作者: William Fedus, Barret Zoph, Noam Shazeer
期刊和年份: JMLR 2022, arXiv 初版 2021
本地原文 PDF: learning/moe-architecture/paper/01_switch_transformers.pdf
本地代码入口:
learning/moe-architecture/src/switch_router.py
learning/moe-architecture/src/common.py
learning/moe-architecture/src/moe_layer_naive.py
learning/moe-architecture/src/mini_moe.py

0. 这篇论文为什么重要

Switch Transformer 是现代稀疏 MoE 语言模型的关键节点。它把早期复杂的 Mixture-of-Experts routing 简化成 top-1 routing: 每个 token 只送到一个 expert。这个简化看起来大胆，因为早期 MoE 认为 top-2 或 top-k routing 更有利于 router 学习；但 Switch 证明，top-1 反而能在质量、速度、稳定性和实现复杂度之间取得更好的工程平衡。

它的核心不是“参数越多越好”这么粗糙，而是：

用稀疏激活增加总参数量，同时让每个 token 的 FFN 计算量大致不变。

Dense Transformer 的每个 token 都经过同一套 FFN 参数。Switch Transformer 把 FFN 换成多个 expert FFN，router 根据 token hidden state 选择一个 expert:

```

token hidden state
    |
    v
router softmax over experts
    |
    v
top-1 expert id
    |
    v
only that expert FFN runs

```

所以它扩展的是“可用参数库”，不是让每个 token 都用全部参数。这正是后来 Mixtral、GLaM、ST-MoE、DeepSeekMoE、Qwen-MoE 等模型的底层思想之一。

1. 当时的技术语境

Dense scaling 已经显示出模型参数、数据和算力的 power-law 改善趋势。但是 dense 模型有一个直接成本：参数更多通常意味着每个 token 的 FLOPs 更多。模型越大，训练和推理越贵。

MoE 给出第四条 scaling axis：在每个 token 计算量近似固定的情况下增加参数量。直觉是，模型有很多 expert 参数，但每个输入只激活其中一小部分。

早期 MoE 难以普及，主要有三个原因：

1. routing 复杂，top-k dispatch 和 combine 难写、难调。
2. 专家分布不均会导致部分 expert 过载，部分 expert 闲置。
3. 大规模分布式训练需要 all-to-all 通信，容易不稳定。

Switch 的设计哲学是尽量把 MoE 做简单：

```

top-k MoE          -> top-1 Switch
多个 expert 输出加权 -> 一个 expert 输出乘 gate
更大 expert capacity -> 更小 capacity
复杂 routing       -> 更低通信和更低计算

```

2. Switch layer 放在哪里

标准 Transformer block 有 self-attention 和 FFN。Switch Transformer 主要替换 FFN，不替换 attention。

Dense Transformer block:

```

x
-> self-attention
-> dense FFN
-> output

```

Switch Transformer block:

```

x
-> self-attention
-> router
    token 0 -> expert 3 FFN

```

```

token 1 -> expert 0 FFN
token 2 -> expert 3 FFN
token 3 -> expert 1 FFN
-> combine
-> output

```

注意, expert 通常也是普通 FFN, 只是每个 expert 有自己独立的一套 FFN 参数。Switch 没有让所有 expert 都跑一遍, 它只跑被选中的那个。

3. Router 数学

给定 token hidden state x , router 是一个线性层:

$$h(x) = W_r x$$

它输出每个 expert 的 logit, 再做 softmax:

$$p_i(x) = \frac{\exp(h_i(x))}{\sum_j \exp(h_j(x))}$$

传统 MoE 会选 top-k experts, 并把多个 expert 的输出按 gate 加权:

$$y = \sum_{i \in T} p_i(x) E_i(x)$$

Switch 把 k 固定为 1。也就是:

```

expert_id = argmax_i p_i(x)
gate = max_i p_i(x)
y = gate * E_expert_id(x)

```

这就是 learning/moe-architecture/src/switch_router.py 的核心:

```

logits = self.W(x)
gates_all = F.softmax(logits, dim=-1)
top1_gate, top1_idx = gates_all.max(dim=-1, keepdim=True)
aux = load_balance_loss(gates_all, top1_idx, self.n_experts)
return top1_gate, top1_idx, aux

```

top-1 routing 的反直觉之处在于: argmax 本身不可导, 但 gate probability 仍然来自 softmax, load balancing loss 也通过平均 router probability 给 router 提供梯度。论文指出, top-1 preserves model quality, 同时减少 routing computation 和 communication。

4. 为什么 top-1 能更高效

top-2 MoE 中, 每个 token 要送到两个 experts, 两个 FFN 都要计算, 再把输出加权合并。

Switch top-1 中, 每个 token 只送到一个 expert。收益有三点。

第一, 计算减少。每个 token 只跑一个 expert FFN。

第二, expert capacity 可以更小。top-2 时每个 token 占两个 expert 的 slot; top-1 时只占一个 slot, 所以每个 expert 的 batch buffer 至少可以减半。

第三, 通信和实现简化。分布式 MoE 的难点是把 token dispatch 到 expert 所在设备, 再把结果 gather 回来。top-1 让 dispatch/combine tensor 更简单。

不过 top-1 也带来风险: 如果 router 把太多 token 分给同一个 expert, 该 expert 的 capacity 会溢出, 部分 token 不能被这个 expert 处理。

5. Expert capacity 和 overflow

论文中每个 expert 的 capacity 是固定的, 因为 TPU/Mesh TensorFlow 需要静态 shape。公式是:

$$\text{expert capacity} = \frac{\text{tokens per batch}}{\text{number of experts}} \times \text{capacity factor}$$

直觉图:

batch has 16 tokens, 4 experts, capacity factor = 1.25

ideal tokens per expert = 16 / 4 = 4

expert capacity = 4 * 1.25 = 5

expert 0 receives 3 tokens -> ok, 2 empty slots

expert 1 receives 5 tokens -> ok

expert 2 receives 7 tokens -> 2 overflow tokens

expert 3 receives 1 token -> many empty slots

capacity factor 越大, overflow 越少, 但 padding/empty slots 越多, 计算和通信浪费越大。capacity factor 越小, 效率越高, 但对 router load balance 要求更高。

论文中 overflow token 的处理方式是: 如果 expert 已满, token 不经过该 Switch FFN, 而是通过 residual connection 进入下一层。这个设计简单, 但过多 dropped tokens 会伤害训练。

仓库里的 `common.capacity` 对应这个概念:

```
def capacity(n_tokens, n_experts, top_k, factor=1.25):  
    return int(factor * n_tokens * top_k / n_experts)
```

对 Switch top-1, `top_k=1`。对 top-2 MoE, `top_k=2`。

6. Load balancing loss

如果没有约束, router 可能把大量 token 分给少数 experts, 形成 expert collapse。Switch 加了 auxiliary load balancing loss。

定义:

- N 是 expert 数量。
- T 是 batch 中 token 数。
- f_i 是实际 dispatch 到 expert i 的 token fraction。
- P_i 是 router probability 在 expert i 上的平均质量。

实际 dispatch fraction:

$$f_i = \frac{1}{T} \sum_{x \in B} 1\{\text{argmax } p(x) = i\}$$

平均 router probability:

$$P_i = \frac{1}{T} \sum_{x \in B} p_i(x)$$

辅助损失:

$$loss = \alpha \cdot N \sum_i f_i P_i$$

论文使用 $\alpha = 10^{-2}$ 。直觉上, 我们希望 f 和 P 都接近均匀分布, 也就是每个 expert 约 $1/N$ 。 f_i 来自 argmax , 不可导; P_i 是 softmax probability 的平均, 可导。所以这个 loss 可以通过 P 给 router 梯度信号。

仓库里的 `common.load_balance_loss` 是去掉 α 的核心形式:

```
n_tokens = gates.shape[0]
f = expert_load(top_k_idx, n_experts) / n_tokens
p = gates.mean(dim=0)
return n_experts * (f * p).sum()
```

训练时通常会写成:

```
loss = main_loss + 0.01 * aux_loss
```

这和论文的 $\alpha = 10^{-2}$ 对应。

7. Switch layer 的数据流

把一个 batch 的 token 走一遍:

```
input hidden states: [tokens, d_model]
```

1. router logits:
[tokens, d_model] @ [d_model, n_experts]
-> [tokens, n_experts]
2. router probs:
softmax over n_experts
3. top-1:
expert_idx: [tokens, 1]
expert_gate: [tokens, 1]
4. dispatch:

```
group token hidden states by expert
each expert receives at most expert_capacity tokens
```

5. expert FFN:
expert e processes only its assigned tokens
6. combine:
place expert outputs back to original token positions
multiply by expert_gate
7. residual path:
tokens dropped by overflow rely on residual connection

这个过程从模型角度看很简单，但从系统角度看很难，因为 token 要跨设备移动。Switch 的 top-1 routing 直接减少了跨设备 token 流量。

8. 训练稳定技巧

Switch 论文不是只提出结构，还给了训练技巧。没有这些技巧，大规模 MoE 很容易不稳定。

第一，selective precision。bfloat16 训练快，但 router softmax 对数值更敏感。论文做法是只把 router 内部计算 cast 到 float32，其他部分保持 bfloat16。这样局部获得 float32 稳定性，又不会让 all-to-all 通信传大块 float32 tensor。

论文 Table 2 显示，纯 bfloat16 的 Switch-Base 会 diverge，而 selective precision 接近 float32 质量，同时保持 bfloat16 速度。

第二，减小初始化尺度。论文建议把默认 Transformer initialization scale 从 1.0 降到 0.1。Table 3 中，0.1x init 的 early training 质量和方差明显更好。

第三，expert dropout。fine-tuning 小数据集时，Switch 有很多参数，更容易过拟合。论文发现不是把所有层 dropout 都调大，而是在非 expert 层保持较小 dropout，比如 0.1，在 expert FFN 内使用较大 dropout，比如 0.4，效果更好。

这些技巧的共同点是：稀疏模型不是只加 experts 就完了，router 和 expert 参数会引入新的训练病态。

9. 实验证据链

Switch 的实验链条很完整。

第一，和 dense T5、top-2 MoE 对比。Table 1 中，Switch-Base 和 MoE-Base 都使用 128 experts，并与 T5 baselines 在同硬件、同训练步数下比较。Switch 在 speed-quality trade-off 上更好。capacity factor 1.0 的 Switch-Base 速度达到约 1000 examples/sec，比 MoE 和 dense T5-Base 的对应方案更有利，达到质量阈值的时间约 62.8 小时。

第二，专家数 scaling。Figure 4 显示，在 FLOPs/token 固定时，专家数从 2、4、8 一直扩到 256，参数量增加，test loss 持续改善。Switch-Base 64 experts 在约 60k steps 达到 T5-Base 约 450k steps 的表现，约 7.5x sample efficiency。

第三，wall-clock speed。Figure 5 显示，Switch-Base 64 experts 在相同计算资源下，用约七分之一时间达到 T5-Base 类似质量，论文称约 7x speedup。Figure 6 进一步和更大的 dense T5-Large 比，T5-Large 每 token FLOPs 约 3.5x，但 Switch-Base 仍有约 2.5x speedup。

第四，downstream fine-tuning。Table 5 中，FLOP-matched Switch-Base/Switch-Large 在多数任务上超过 T5-Base/T5-Large。例子包括 SuperGLUE: T5-Base 75.1, Switch-Base 79.5; T5-Large 82.7,

Switch-Large 84.7。闭卷 TriviaQA 也有明显提升: T5-Base 24.5, Switch-Base 30.7; T5-Large 29.5, Switch-Large 36.9。

第五, distillation。大稀疏模型不易部署, 所以论文把 Switch teacher 蒸馏到 dense T5-Base。Table 6/7 显示, 即使把大 sparse teacher 压缩 95% 以上, 仍能保留约 30% 的质量提升。SuperGLUE 上 7.4B Switch-Base 蒸馏回 223M T5-Base, 可在 97% 压缩下保留约 30% teacher gain。

第六, 多语言。mSwitch-Base 在 101 种语言上相对 mT5-Base 都有改善, 平均 speedup 约 5x, 91% 语言达到至少 4x speedup。

第七, 万亿参数。论文组合 data/model/expert parallelism, 训练 395B Switch-XXL 和 1.571T Switch-C。Table 9 显示, Switch-C 在同 compute budget 下达到固定 perplexity 比 T5-XXL 约 4x 更快。Switch-XXL 在 500k steps 时 C4 pretraining quality 优于 T5-XXL, 但 downstream 转化并不总是完全同步, 这也是论文承认的开放问题。

10. 为什么参数多但 FLOPs 不同步增加

Dense FFN:

```
each token uses one huge shared FFN
parameters used per token = all FFN parameters
FLOPs per token grows with FFN size
```

Switch FFN:

```
there are many expert FFNs
each token uses one selected expert
parameters available to model = sum over all experts
FLOPs per token roughly = one expert FFN + router
```

所以增加 experts 会增加总参数, 但每个 token 只用一个 expert, FLOPs/token 基本不随 expert 数线性增长。router 的额外成本是 $O(d_{model} \times n_{experts})$, 相对 FFN 成本通常较小, 但当 expert 数非常大、系统通信复杂时, 它也不是零成本。

这也是 MoE 的根本 trade-off:

```
more experts
-> more parameters
-> better sample efficiency
-> more routing/communication/memory complexity
```

11. 本地代码怎么对应论文

switch_router.py 对应论文 top-1 Switch routing:

```
x -> Linear(d_model, n_experts) -> softmax -> max -> top1 expert
```

common.py 对应三个基础工具:

```
expert_load:
    统计每个 expert 收到多少 token
```

```
load_balance_loss:
    实现  $N * \sum(f_i * p_i)$ 
```

capacity:

实现 $\text{capacity_factor} * \text{tokens} * \text{top_k} / \text{n_experts}$

moe_layer_naive.py 是 top-k MoE 教学层。它默认 top_k=2, 更接近早期 MoE/GShard, 不是 Switch 的 top-1, 但能帮助你比较 top-2 与 top-1 的区别。

mini_moe.py 是更现代的 capstone, 包含 AuxFreeRouter 和 shared expert 思想, 已经超出 Switch 论文。学习顺序应该是:

switch_router.py
-> top-1 routing

common.py
-> load balance and capacity

moe_layer_naive.py
-> top-k MoE comparison

mini_moe.py
-> modern MoE extension

一个最小 Switch forward 可以这样写:

```
import torch
from switch_router import SwitchRouter

router = SwitchRouter(d_model=16, n_experts=4)
x = torch.randn(20, 16)

gate, expert_idx, aux = router(x)

print(gate.shape)          # [20, 1]
print(expert_idx.shape)    # [20, 1]
print(aux.item())
```

你应该手动检查:

```
torch.bincount(expert_idx.flatten(), minlength=4)
```

如果某个 expert 的 count 远高于其他 expert, 就说明 load balancing 还没有做好。

12. 与后来 MoE 的关系

Switch 是很多现代 MoE 的地基, 但不是终点。

GShard 使用 top-2 routing 和更复杂的负载均衡。

Switch 使用 top-1 routing, 简单高效。

ST-MoE 后来加入 router z-loss 等稳定技巧。本仓库 router_z_loss.py 对应的是后续思想, 不属于 Switch 原始论文主体。

Mixtral 使用 top-2 routing, 并在 decoder-only LLM 中大规模应用 MoE。

DeepSeekMoE/DeepSeek-V3 一类方法进一步引入 shared experts、fine-grained experts、aux-loss-free balancing 等设计。本仓库 mini_moe.py 和 aux_loss_free.py 更接近这些后续方向。

学习上不要把所有 MoE 都混成一个词。Switch 的独特位置是：

用 top-1 routing 证明 MoE 可以被极大简化，并仍然稳定有效地扩到百亿、千亿、万亿参数。

13. 局限和开放问题

第一，training stability 仍然困难。论文明确说，稳定技巧对 Switch-Base、Switch-Large、Switch-C 有效，但对 Switch-XXL 还不够。

第二，pretraining perplexity 不总能完美转化为 downstream。论文提到 1.6T Switch-C 在某些 downstream 上不如更小但 FLOPs/token 更高的 Switch-XXL，说明参数量、FLOPs/token 和 fine-tuning 之间关系还不清楚。

第三，communication cost 很真实。MoE 的 FLOPs/token 看起来低，但跨设备 all-to-all 可能成为瓶颈。

第四，expert collapse 需要持续管理。load balancing loss、capacity factor、router precision、dropout、初始化都在围绕这个问题服务。

第五，Switch 的 sparsity 是 FFN expert sparsity，不是 attention sparsity。它不能替代 FlashAttention、long-context attention 或 KV cache 优化。

14. 对今天的意义

今天很多 LLM 采用 MoE，是因为 Switch 这类工作把“参数量”和“每 token 计算量”拆开了。它让人们看到，可以把模型变成一个很大的 expert 参数库，而每个 token 只调用一小部分。

Switch 对学习者最重要的意义是建立 MoE 的基础坐标：

- router 是决策器。
- expert 是条件激活的 FFN 参数。
- top-k 决定每个 token 用几个专家。
- capacity 决定每个 expert 最多接多少 token。
- overflow/dropped token 是稀疏路由的真实代价。
- load balancing loss 防止 router 偏向少数专家。
- all-to-all communication 是系统瓶颈。

你以后读 Mixtral、DeepSeekMoE、Qwen-MoE、GLaM、ST-MoE 或 MegaBlocks，都可以先问：它和 Switch 相比，改的是 router、expert granularity、load balancing、dispatch kernel、通信，还是训练稳定性？

15. 常见误解

误解一：Switch 每个 token 使用万亿参数。

不是。模型总参数可以到万亿，但每个 token 只激活一个 expert 的 FFN 参数。

误解二：top-1 routing 一定比 top-2 表达能力弱。

不一定。论文表明 top-1 在质量、速度和稳定性上整体更优，尤其是大规模训练时。

误解三：MoE 的效率只看 FLOPs。

不行。all-to-all 通信、capacity padding、dropped tokens、router 开销都必须算。

误解四：load balancing loss 只是装饰。

不是。没有它，router 可能 collapse 到少数 experts，导致 overflow 和参数浪费。

误解五: Switch 解决了所有 MoE 问题。

没有。它简化了 routing 并展示大规模可行性, 但稳定性、downstream 转化、通信优化和专家异质性仍是开放问题。

16. 闭卷检查

读完后你应该能回答:

1. Switch Transformer 替换的是 Transformer 的哪一部分?
2. top-1 routing 和 top-2 MoE 的计算差异是什么?
3. router logits、router probability、expert gate、expert index 分别是什么?
4. expert capacity 怎么计算?
5. capacity factor 变大和变小分别有什么代价?
6. overflow token 在论文中如何处理?
7. f_i 和 P_i 在 load balancing loss 中分别表示什么?
8. 为什么 f_i 不可导但 loss 仍能训练 router?
9. selective precision 为什么只把 router 部分 cast 到 float32?
10. expert dropout 为什么只增强 expert 内部 dropout, 而不是全模型 dropout?
11. Switch 为什么能增加总参数但保持 FLOPs/token 近似固定?
12. 为什么 pretraining perplexity 提升不一定完全转化为 downstream 提升?

17. 用 AI agent 学这篇的正确方式

第一轮, 让 agent 画 token dispatch:

请用 12 个 tokens、3 个 experts、capacity_factor=1.0 的例子, 给出 router probability、top-1 expert、每个 expert 的 capacity、overflow token, 以及最终哪些 token 被 expert 处理。

第二轮, 对照代码:

请逐行解释 learning/moe-architecture/src/switch_router.py 和 common.py。重点说明 gates_all、top1_gate、top1_idx、f、p、load_balance_loss 的张量形状。

第三轮, 做概念对比:

请比较 Switch top-1、GShard top-2、Mixtral top-2、DeepSeekMoE shared expert。每种只讲 routing 和 expert 激活方式, 不要泛泛总结。

第四轮, 反向考试:

请问我 10 个 MoE 闭卷题。每当我把参数量、FLOPs/token、通信成本、capacity、load balance 混在一起时, 请指出具体混淆点。

最后要能闭卷说出:

Switch Transformer 的关键是用 top-1 sparse FFN expert routing, 把总参数量扩成很多 experts, 但让每个 token 只计算一个 expert, 再用 capacity 和 load balancing loss 管住路由不均衡。